

VirPet

Reigns tarzı sanal evcil hayvan oyunu:
Java Swing masaüstü uygulaması ve Android portu

Öğrenci Adı Soyadı Emine Mihrinaz Kurt

Öğrenci Numarası 24040102017

Proje Adı VirPet (Virtual Pet)

Dil / Platform Java 8+, Swing; Android (Gradle)

Ek `README.md`, `manual.html`
Dokümantasyon

Haziran 2025

Özet

VirPet, Reigns benzeri kart tabanlı karar verme mekaniğini sanal evcil hayvan bakımıyla birleştiren bir Java projesidir. Oyuncu önce şapka, vücut, yüz ve bacak parçalarından oluşan bir pet oluşturur; ardından rastgele gelen olay kartlarına *Yes* veya *No* yanıtı vererek dört istatistiği (açlık, sevgi, kilo, sağlık) dengede tutmaya çalışır. Her turda pasif zaman geçişi statları otomatik değiştirir; sağlık veya sevgi sıfıra inince oyun biter.

Proje, iş mantığını arayüzden ayıran modüler bir sınıf yapısı (`PetGameModel`, olay kartı sistemi, sprite birleştirici) kullanır. Olay içerikleri kod dışında `event_cards.cards` dosyasında tutulur; böylece yeni kartlar derleme gerektirmeden eklenebilir. Aynı oyun mantığı `androidPort/` altında tek Activity dosyasıyla Android'e taşınmıştır. Bu rapor projenin genel dokümantasyonunu, mimarisini ve çalışma prensiplerini özetler.

İçindekiler

1. Giriş
2. Proje Tanımı ve Amaç
3. Oyun Mekaniği
4. Yazılım Mimarisi
5. Sınıf Yapısı ve Modüller
6. Varlık Sistemi
7. Platformlar
8. Kurulum ve Çalıştırma
9. Kullanılan Teknolojiler
10. Sonuç

1. Giriş

Sanal evcil hayvan oyunları, kullanıcıların düzenli etkileşim ve kaynak yönetimiyle bağ kurmasını sağlayan klasik bir türdür. **VirPet** bu türü, mobil oyun *Reigns*'in sade kart karar mekaniğiyle birleştirir: her turda tek bir olay metni ve iki seçenek sunulur; seçimler görünürde basit olsa da dört sayısal göstergenin uzun vadeli dengesini etkiler.

Proje iki hedefe yöneliktir:

- **Oynanabilir bir ürün:** Pet oluşturma ekranı, oyun ekranı, stat çubukları, piksel sanat sprite'ları ve ses geri bildirimi.
- **Öğretici kod yapısı:** Model-görünüm ayırımına yakın düzen, dosyadan veri okuma, enum kullanımı, `java.nio.file.Path` ile kaynak yönetimi ve isteğe bağlı Android portu.

Detaylı satır satır kod açıklamaları `manual.html` dosyasında; bu rapor ise projenin bütünsel teknik dokümantasyonunu sunar.

2. Proje Tanımı ve Amaç

2.1 Proje özeti

VirPet, masaüstünde Java Swing ile çalışan pencere tabanlı bir uygulamadır. Oyuncu akışı iki ana fazdan oluşur:

- Oluşturma (creation):** Şapka, vücut stili, yüz stili ve bacak parçaları ok tuşlarıyla seçilir; pet'e isim verilir ve önizleme panelinde canlı sprite gösterilir.
- Oyun (game):** *Start* ile oyun ekranına geçilir; arka plan müziği, olay kartları ve Yes/No düğmeleri devreye girer.

2.2 Tasarım hedefleri

Hedef	Uygulama
Genişletilebilir içerik	44 olay kartı harici <code>.cards</code> dosyasında; yeni kart eklemek için yalnızca metin düzenlemek yeterli
Ayrık iş kuralları	<code>PetGameModel</code> stat, zaman ve kaybetme mantığını UI'dan izole eder
Görsel geri bildirim	Kilo ve sevgiye göre dinamik vücut/yüz PNG seçimi; katmanlı 16×48 sprite
Çoklu platform	Masaüstü (WAV + Swing) ve Android (MP3 + View/Activity)

2.3 Proje dizin yapısı

```
virPet/
├─ PetCreationApp.java      Ana arayüz ve oyun döngüsü
├─ PetGameModel.java        Stat kuralları, zaman, kaybetme
├─ PetSpriteComposer.java   Sprite birleştirme
├─ EventCard.java           Kart veri modeli
├─ StatDelta.java           Seçim etkisi (4 stat)
├─ EventCardLoader.java     Kart dosyası okuyucu
├─ EventCardDeck.java       Rastgele kart dağıtımı
├─ GameAudio.java           Masaüstü ses (WAV)
├─ assets/                  Görseller, ses, font, kartlar
├─ androidPort/             Gradle Android projesi
├─ manual.html              Kod el kitabı (Türkçe)
├─ README.md                Hızlı başlangıç
└─ rapor.html               Bu belge
```

3. Oyun Mekaniği

3.1 İstatistikler (stats)

Dört gösterge 0-100 aralığında tutulur (`STAT_MIN` , `STAT_MAX`). Başlangıç değerleri: sağlık 100, sevgi 70, açlık 30, kilo 50.

Stat	Anlam	Risk
Hunger (açlık)	Yüksek = pet aç	Dolaylı; aşırı kilo/sağlık etkileşimi
Affection (sevgi)	Düşük = mutsuz yüz	0 → pet evi terk eder (<code>LEFT_HOME</code>)
Weight (kilo)	Görsel gövde tipi	<25 veya >75 iken her tur sağlık -2
Health (sağlık)	Genel yaşam	0 → oyun biter (obezite veya açlık mesajı)

3.2 Olay kartları ve seçimler

Her turda `EventCardDeck` bir sonraki kartı verir. Oyuncu *Yes* veya *No* seçince ilgili `StatDelta` uygulanır; ardından otomatik **zaman geçişi** çalışır. Kart metinleri İngilizcedir; örnek format:

```
card empty_bowl
text
The food bowl is empty. Your pet looks at you with big eyes. Fill it up?
yes hunger=-14 affection=3 weight=6
no hunger=6 affection=-6
end
```

Dosyada toplam **44** kart tanımlıdır. `#` ile başlayan satırlar ve boş satırlar yok sayılır.

3.3 Pasif zaman geçişi

Her `applyChoice` sonrası `applyTimePassage()` şu sabitleri uygular:

- Açlık: **+3**
- Kilo: **-2**
- Sevgi: **-1**
- Kilo ≤ 25 veya ≥ 75 ise sağlık: **-2**

Tüm değerler `clamp()` ile 0-100 arasında sınırlandırılır.

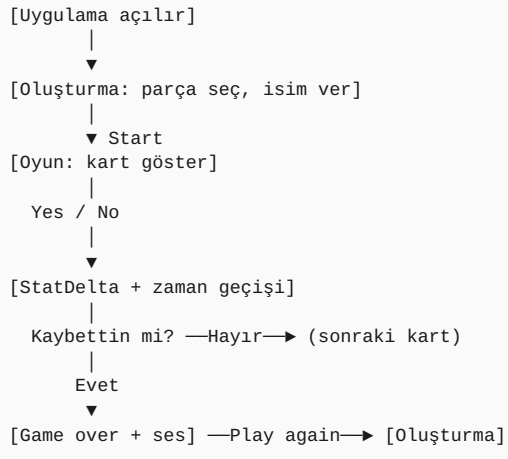
3.4 Kaybetme koşulları

`GameOverReason` enum'u üç sonuç tipi tanımlar:

Enum	Koşul	Kullanıcı mesajı (örnek)
<code>LEFT_HOME</code>	Sevgi ≤ 0	«... left the house.»
<code>OBESITY</code>	Sağlık ≤ 0 , yüksek kilo	«... died of obesity.»
<code>STARVATION</code>	Sağlık ≤ 0 , düşük kilo	«... starved to death.»

Game over sonrası *Play again* oluşturma ekranına döner; model ve deste yeniden başlatılır.

3.5 Oyun akış diyagramı



řekil 1 — VirPet oyun dngüsü (basitleřtirilmiř)

4. Yazılım Mimarisi

4.1 Katmanlar

Mimari fiilen üç katmana ayrılır:

1. **Sunum:** `PetCreationApp` — Swing panelleri, düğmeler, önizleme çizimi, kart metni, stat çubukları.
2. **İş mantığı:** `PetGameModel` — stat güncelleme, sprite yolu çözümü, kaybetme değerlendirme.
3. **Veri / altyapı:** `EventCardLoader`, `EventCardDeck`, `PetSpriteComposer`, `GameAudio`, `assets/`.

4.2 Veri akışı (bir tur)



4.3 Arayüz düzeni (masaüstü)

`CardLayout` ile iki tam ekran panel arasında geçiş yapılır: oluşturma ve oyun. Oyun panelinde pet görüntüsü, dört stat çubuğu, kaydırılabilir olay metni ve Yes/No düğmeleri yer alır. Global fare tıklamaları `GameAudio` ile kısa tap sesi çalar.

5. Sınıf Yapısı ve Modüller

Sınıf	Sorumluluk	Önemli kavramlar
StatDelta	Bir seçimin hunger/affection/weight/health deltasını taşır	public static final ZERO sabit nesne
EventCard	id, metin, yes/no deltası	Encapsulation, immutable alanlar
EventCardLoader	event_cards.cards parse	try-with-resources, BufferedReader, satır durum makinesi
EventCardDeck	Karışık sıra, ardışık aynı kartı önleme	Collections.shuffle, indeks yönetimi
PetGameModel	Statlar, zaman, kaybetme, PNG yolu	enum, Path.resolve, clamp
PetSpriteComposer	16×48 ARGB tuval, katman çizimi	BufferedImage, Graphics2D, ImageIO
GameAudio	bg / tap / death / runaway WAV	javax.sound.sampled.Clip
PetCreationApp	UI, olay döngüsü, asset yükleme	Swing, CardLayout, özel JPanel alt sınıfları

5.1 PetGameModel — eşik değerleri

Sabit	Değer	Kullanım
WEIGHT_SKINNY	35	Altında skinny gövde PNG
WEIGHT_FAT	65	Üstünde fat gövde PNG
AFFECTION_SAD / HAPPY	35 / 65	Sad / happy yüz PNG
WEIGHT_HEALTH_LOW / HIGH	25 / 75	Ekstra sağlık kaybı bandı

5.2 EventCardDeck — tekrar önleme

Deste her turda karıştırılabilir; bir kart çekildikten hemen sonra aynı kart tekrar gelmez (son çekilen id saklanır). Bu, ardışık tekrarlayan olayların oyuncu deneyimini bozmasını azaltır.

6. Varlık Sistemi

6.1 Sprite katmanları

Pet görüntüsü dört PNG parçasının 16×48 piksel tuval üzerine alttan üste birleştirilmesiyle oluşur:

Katman	Y konumu	Seçim kriteri
Bacak	32	Oluşturmada seçilen <code>legsN.png</code>
Gövde	16	Stil id + kilo → skinny / normal / fat
Yüz	16	Stil id + sevgi → sad / normal / happy
Şapka	0	<code>hats/hatN.png</code>

Parça dosyaları `assets/parts/` altında tutulur; üç vücut stili (0-2) ve iki yüz stili (0-1) desteklenir.

6.2 Ses ve görsel tema

- **Font:** `DS-TERM.TTF` — retro terminal görünümü
- **Arka plan:** `bg.png` (2730×1536); önizleme panelleri en-boy oranını korur
- **Ses (masaüstü):** `bg.wav`, `tap.wav`, `death.wav`, `runaway.wav`
- **Ses (Android):** Aynı içerik MP3 olarak `app/src/main/assets/`

6.3 Olay kartı dosya formatı

Anahtar kelimeler: `card`, `text`, `yes`, `no`, `end`. Delta satırları `stat=değer` çiftleri içerir (ör. `hunger=-14`). Yazılmayan statlar için `StatDelta.ZERO` kullanılır.

7. Platformlar

7.1 Masaüstü (Java Swing)

Ana giriş noktası `PetCreationApp.main`. Derleme ve çalıştırma proje kökünden yapılır; tüm `.java` dosyaları birlikte derlenmelidir (sürüm uyumsuzluğu önlenir).

7.2 Android portu

`androidPort/` ayrı bir Gradle projesidir. Oyun mantığı `VirPetActivity.java` içinde tek dosyada toplanmıştır; masaüstü sınıflarıyla paralel stat ve kart kuralları uygulanır. Farklar:

Özellik	Masaüstü	Android
Arayüz	Swing (<code>JFrame</code> , paneller)	Activity + özel <code>View</code> çizimi
Ses	<code>javax.sound.sampled</code> (WAV)	<code>MediaPlayer</code> (MP3)
Dosya	<code>java.nio.file</code> , çalışma dizini <code>assets/</code>	<code>AssetManager</code> , APK içi assets
Derleme	<code>javac</code> + <code>java</code>	<code>./gradlew assembleDebug</code>

Asset güncellemesi: masaüstü `assets/` değişince Android tarafına kopyalanmalıdır (`cp -r ../assets/* app/src/main/assets/`).

8. Kurulum ve Çalıştırma

8.1 Gereksinimler

- **Masaüstü:** JDK 8 veya üzeri; ek ses kütüphanesi gerekmez.
- **Android:** Android SDK, JDK 17 (Gradle 8.x), Android Studio önerilir.

8.2 Masaüstü komutları

```
cd virPet
javac -source 8 -target 8 *.java
java PetCreationApp
```

Komutlar proje kök dizininden çalıştırılmalıdır; `assets/` görelî yol olarak okunur.

8.3 Android komutları

```
cd androidPort
export ANDROID_HOME=~/Android/Sdk
./gradlew assembleDebug
```

Çıktı APK: `app/build/outputs/apk/debug/app-debug.apk`

8.4 İlgili belgeler

- `README.md` — hızlı özet ve kart formatı
- `manual.html` — sınıf sınıf kod el kitabı, Mermaid diyagramları, Android detayı
- `rapor.html` — bu proje raporu (PDF yazdırmaya uygun)

9. Kullanılan Teknolojiler

Alan	Masaüstü	Android
Dil	Java 8+	Java (Android SDK)
GUI	Java Swing	Android View sistemi
Görüntü	AWT <code>BufferedImage</code> , <code>ImageIO</code>	<code>Bitmap</code> , <code>Canvas</code>
Ses	<code>javax.sound.sampled</code>	<code>MediaPlayer</code>
Dosya I/O	<code>java.nio.file</code>	<code>AssetManager</code>
Derleme	<code>javac</code>	Gradle Kotlin DSL (<code>build.gradle.kts</code>)

9.1 Öne çıkan Java kavramları

Projede pedagojik olarak şu yapılar kullanılmıştır: **enum** (`GameOverReason`), **immutable veri sınıfları**, **static factory/loader** metotları, **try-with-resources**, **Path API**, **inner class** (önizleme panelleri), **event listener** (Swing fare/ pencere olayları).

10. Sonuç

VirPet, kart tabanlı karar verme ile sanal pet bakımını bir araya getiren, modüler ve genişletilebilir bir Java projesidir. İş kuralları `PetGameModel` ve harici kart dosyasında merkezileştirilmiş; görsel geri bildirim kilo ve sevgi eşiklerine bağlanmıştır. Masaüstü sürüm tam özellikli bir Swing deneyimi sunarken, Android portu aynı tasarımın mobil dağıtımını gösterir.

Gelecekte kart sayısının artırılması, yeni parça setleri, kayıt/yükleme (save game) veya lokalizasyon (Türkçe olay metinleri) düşük maliyetli geliştirmeler olarak öne çıkar; mevcut mimari bu eklemelere uygun ayırım sağlar.